МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Кафедра информатики и вычислительной техники

«Ветвления и циклы в Python»
Отчет по лабораторной работе № 2
по дисциплине «Программирование на Python»

Выполнил студент группы ИВТ-б-о-25-1
« » октября 2026г.
Подпись студента_____
Работа защищена « »_____20_26_г.
Проверила _____
(подпись)

Ставрополь 2026

Ссылка на репозиторий: https://github.com/DJOK2/laba1.git (папка лаба2)

Тема: Ветвления и циклы в Python.

Цель работы: освоить выбор действий с помощью if-elif-else и простейшего match-case, перебор с помощью for и range(), циклы while, операторы break и continue; научиться проверять границы условий и завершение циклов.

Окружение: ОС Windows 10 Home (10.0.19045), Python 3.14.4, редактор VS Code.

Назначенные варианты: группа 1 — вариант 13, группа 2 — вариант 13, группа 3 — вариант 13, группа 4 — вариант 13.

Ход работы

Индивидуальное задание 1 (группа 1, вариант 13)

Условие. Студенты убирают урожай помидоров. При сборе от 0 до 50 кг включительно в день работа оплачивается из расчёта 30 коп. за 1 кг; при сборе свыше 50 до 75 кг включительно — 50 коп. за 1 кг; при сборе свыше 75 до 90 кг включительно — 65 коп. за 1 кг; при сборе свыше 90 кг в день — 70 коп. за 1 кг плюс 20 руб. премия. Студент собрал X кг за день. Определить его заработок ($0 \le X \le 1000$); ставка применяется ко всему собранному весу, премия добавляется один раз.

Алгоритм. Вводится вес X (float). Если X вне диапазона от 0 до 1000, сообщение об ошибке выводится в stderr и программа завершается с кодом 1. Иначе цепочкой if-elif-else выбирается ставка (30, 50, 65 или 70 коп.) по верхним границам диапазонов 50, 75 и 90 кг (границы относятся к нижнему тарифу). Премия 20 руб. добавляется один раз только при $X > 90$. Заработок = X · ставка / 100 + премия, выводится с двумя знаками после точки. Порядок условий от меньшего к большему гарантирует, что каждое значение попадает ровно в одну ветвь.

Листинг программы individual_01.py:

```
import sys


def daily_pay(kg):
```

```
    if kg <= 50:
        rate = 30
    elif kg <= 75:
        rate = 50
    elif kg <= 90:
        rate = 65
    else:
        rate = 70

    bonus = 20 if kg > 90 else 0
    return kg * rate / 100 + bonus


if __name__ == "__main__":
    x = float(input("Введите вес собранных помидоров, кг: "))

    if x < 0 or x > 1000:
        print("Недопустимое значение: вес должен быть от 0 до 1000 кг", file=sys
.stderr)
        sys.exit(1)

    print(f"Заработок за день: {daily_pay(x):.2f} руб.")
```

Таблица 1. Проверки программы individual_01.py

Входные данные (по одному на строку)	**Ожидаемый результат**	**Фактический результат**	**Вывод**
0	Заработок за день: 0.00 руб.	Заработок за день: 0.00 руб.	Совпадает
30	Заработок за день: 9.00 руб.	Заработок за день: 9.00 руб.	Совпадает
50	Заработок за день: 15.00 руб.	Заработок за день: 15.00 руб.	Совпадает
50.5	Заработок за день: 25.25 руб.	Заработок за день: 25.25 руб.	Совпадает
75	Заработок за день: 37.50 руб.	Заработок за день: 37.50 руб.	Совпадает
75.5	Заработок за день: 49.08 руб.	Заработок за день: 49.08 руб.	Совпадает
90	Заработок за день: 58.50 руб.	Заработок за день: 58.50 руб.	Совпадает
90.5	Заработок за день: 83.35 руб.	Заработок за день: 83.35 руб.	Совпадает

1000	Заработок за день: 720.00 руб.	Заработок за день: 720.00 руб.	Совпадает
-1	Ошибка	Ошибка: Недопустимое значение: вес должен быть от 0 до 1000 кг (код завершения 1)	Совпадает
1000.1	Ошибка	Ошибка: Недопустимое значение: вес должен быть от 0 до 1000 кг (код завершения 1)	Совпадает

Значение 75.5 кг: 75.5 · 65 / 100 = 49.075 руб., при выводе с двумя знаками получается 49.08. Значения на границах (50, 75, 90) относятся к нижнему тарифу, значения 50.5, 75.5 и 90.5 — к следующему; при 90.5 кг добавляется премия. Значения -1 и 1000.1 вне допустимого диапазона.

Индивидуальное задание 2 (группа 2, вариант 13)

Условие. Составить программу, выясняющую, делится ли натуральное число x нацело на натуральное число y.

Алгоритм. Вводятся целые x и y. Если одно из них меньше 1, сообщение об ошибке выводится в stderr, код завершения 1 (это же исключает деление на ноль). Иначе вычисляется остаток x % y; если он равен нулю, выводится, что x делится на y нацело, и частное x // y, в противном случае выводится, что не делится, и остаток.

Листинг программы individual_02.py:

```
import sys


if __name__ == "__main__":
    x = int(input("Введите натуральное число x: "))
    y = int(input("Введите натуральное число y: "))

    if x < 1 or y < 1:
        print("Недопустимое значение: числа должны быть натуральными", file=
sys.stderr)
        sys.exit(1)

    remainder = x % y

    if remainder == 0:
        print(f"Число {x} делится на {y} нацело, частное: {x // y}")
    else:
```

print(f"Число {x} не делится на {y} нацело, остаток: {remainder}")

Таблица 2. Проверки программы individual_02.py

Входные данные (по одному на строку)	Ожидаемый результат	Фактический результат	Вывод
12 4	Число 12 делится на 4 нацело, частное: 3	Число 12 делится на 4 нацело, частное: 3	Совпадает
13 4	Число 13 не делится на 4 нацело, остаток: 1	Число 13 не делится на 4 нацело, остаток: 1	Совпадает
5 7	Число 5 не делится на 7 нацело, остаток: 5	Число 5 не делится на 7 нацело, остаток: 5	Совпадает
7 7	Число 7 делится на 7 нацело, частное: 1	Число 7 делится на 7 нацело, частное: 1	Совпадает
1 1	Число 1 делится на 1 нацело, частное: 1	Число 1 делится на 1 нацело, частное: 1	Совпадает
0 5	Ошибка	Ошибка: Недопустимое значение: числа должны быть натуральными (код завершения 1)	Совпадает
6 0	Ошибка	Ошибка: Недопустимое значение: числа должны быть натуральными (код завершения 1)	Совпадает

Проверены оба исхода ветвления (делится и не делится), случай x < y (остаток равен x), равные числа и наименьшие допустимые значения, а также недопустимые x = 0 и y = 0.

Индивидуальное задание 3 (группа 3, вариант 13)

Условие. Дано натуральное число 1 ≤ n ≤ 100000. Получить все его натуральные делители.

Алгоритм. Вводится n и проверяется диапазон. Цикл while перебирает кандидата d от 1 до n включительно, шаг 1. Если n % d == 0, то d добавляется в список делителей. После цикла выводятся количество делителей и сами делители в порядке возрастания. Цикл завершается, потому что d увеличивается на 1 на каждой итерации и рано или поздно становится больше n. Условие d <= n включает правую границу, поэтому сам n попадает в список.

Листинг программы individual_03.py:

```
import sys
```

```
if __name__ == "__main__":
    n = int(input("Введите натуральное число n от 1 до 100000: "))

    if n < 1 or n > 100000:
        print("Недопустимое значение: n должно быть от 1 до 100000", file=sys.st
derr)
        sys.exit(1)

    divisors = []
    d = 1
    while d <= n:
        if n % d == 0:
            divisors.append(d)
        d += 1

    print(f"Количество делителей: {len(divisors)}")
    print("Делители:", *divisors)
```

Таблица 3. Проверки программы individual_03.py

Входные данные (по одному на строку)	**Ожидаемый результат**	**Фактический результат**	**Вывод**
1	Количество делителей: 1 Делители: 1	Количество делителей: 1 Делители: 1	Совпадает
12	Количество делителей: 6 Делители: 1 2 3 4 6 12	Количество делителей: 6 Делители: 1 2 3 4 6 12	Совпадает
13	Количество делителей: 2 Делители: 1 13	Количество делителей: 2 Делители: 1 13	Совпадает
36	Количество делителей: 9 Делители: 1 2 3 4 6 9 12 18 36	Количество делителей: 9 Делители: 1 2 3 4 6 9 12 18 36	Совпадает
100000	Количество делителей: 36 Делители: 1 2 4 5 8 10 16 20 25 32 40 50 80 100 125 160 200 250 400 500 625 800 1000 1250 2000 2500 3125 4000 5000 6250 10000 12500 20000 25000 50000 100000	Количество делителей: 36 Делители: 1 2 4 5 8 10 16 20 25 32 40 50 80 100 125 160 200 250 400 500 625 800 1000 1250 2000 2500 3125 4000 5000 6250 10000 12500 20000 25000 50000 100000	Совпадает
0	Ошибка	Ошибка: Недопустимое значение: n должно быть от 1 до 100000 (код завершения 1)	Совпадает

100001	Ошибка	Ошибка: Недопустимое значение: n должно быть от 1 до 100000 (код завершения 1)	Совпадает

Проверены наименьшее n = 1 (единственный делитель), простое число 13 (два делителя), составные 12 и 36 (у 36 корень 6 входит в список один раз), наибольшее n = 100000 и недопустимые 0 и 100001.

Таблица 4. Трассировка программы individual_03.py при n = 12

d	n % d	Делитель?	Список divisors после шага
1	0	да	[1]
2	0	да	[1, 2]
3	0	да	[1, 2, 3]
4	0	да	[1, 2, 3, 4]
5	2	нет	[1, 2, 3, 4]
6	0	да	[1, 2, 3, 4, 6]
7	5	нет	[1, 2, 3, 4, 6]
8	4	нет	[1, 2, 3, 4, 6]
9	3	нет	[1, 2, 3, 4, 6]
10	2	нет	[1, 2, 3, 4, 6]
11	1	нет	[1, 2, 3, 4, 6]
12	0	да	[1, 2, 3, 4, 6, 12]

Наблюдение в отладчике. Точка останова на строке if n % d == 0:. При первой остановке n = 12, d = 1, divisors = []; остаток равен 0, поэтому следующим шагом выполняется divisors.append(d). При остановке с d = 5 значения n = 12, d = 5, divisors = [1, 2, 3, 4]; остаток равен 2, условие ложно, следующим шагом выполняется d += 1. Последняя итерация проходит при d = 12 (divisors = [1, 2, 3, 4, 6, 12] после добавления), затем d становится 13, условие d <= n ложно и цикл завершается.

Индивидуальное задание 4 (группа 4, вариант 13)

Условие. Вычислить значение гиперболического синуса суммированием ряда с точностью $\varepsilon = 10^{-10}$ (не более 1000 членов), допустимый диапазон $-1 \le x \le 1$:

$$\sinh x = (e^x - e^{-x})/2 = \Sigma\, x^{(2k+1)} / (2k+1)!,\ k = 0, 1, 2, \ldots$$

Исходный ряд и первый член. a_k = x^(2k+1) / (2k+1)!, начальный индекс k = 0, первый член a_0 = x. Постоянных слагаемых вне суммы и множителей перед знаком суммы нет.

Рекуррентное соотношение. $a_{k+1} = a_k \cdot q_k(x)$, где $q_k(x) = a_{k+1}/a_k$ = x^(2k+3)/(2k+3)! · (2k+1)!/x^(2k+1) = x^2 / ((2k+2)(2k+3)). В программе текущий член умножается на этот коэффициент, а не делится на предыдущий, поэтому при x = 0 деление на нуль невозможно, а степени и факториалы на каждой итерации заново не вычисляются.

Оценка остатка и условие остановки. Для $k \geq 0$ и $|x| \leq 1$ имеем $|q_k(x)| \leq 1/(2 \cdot 3) = 1/6 \leq 1/2$, то есть $|a_{k+1}| \leq \frac{1}{2}|a_k|$. Тогда после включения члена a_k остаток $|R_k| \leq |a_k|(1/2 + 1/4 + 1/8 + \ldots) = |a_k|$. Поэтому оценкой остатка служит abs(term) — модуль последнего включённого члена, а цикл while продолжается, пока abs(term) > ε. Цикл конечен: члены убывают как минимум в два раза; кроме того, введён защитный предел MAX_TERMS = 1000, при его достижении выдаётся сообщение в stderr и код завершения 1.

Алгоритм. Вводится x (float); при |x| > 1 — сообщение в stderr и код 1. Задаются первый член term = x, сумма total = x, счётчик count = 1 и индекс k = 0. Пока abs(term) > ε: term *= x^2 / ((2k+2)(2k+3)), k += 1, total += term, count += 1. Затем выводятся значение с десятью знаками после точки, число включённых членов и оценка остатка в научной записи. После основного вычисления выполняется независимый проверочный расчёт: сумма первых 50 членов непосредственно по формуле x^(2k+1)/(2k+1)! с math.factorial и math.fsum, и сравнение с основным результатом и с math.sinh.

Листинг программы individual_04.py:

```
import math
import sys

EPSILON = 1e-10
MAX_TERMS = 1000


def sinh_series(x):
    term = x
```

```
    total = term
    count = 1
    k = 0

    while abs(term) > EPSILON:
        if count >= MAX_TERMS:
            print("Превышен предел числа членов ряда", file=sys.stderr)
            sys.exit(1)

        term *= x * x / ((2 * k + 2) * (2 * k + 3))
        k += 1
        total += term
        count += 1

    return total, count, abs(term)


def sinh_check(x, terms=50):
    parts = []
    for k in range(terms):
        parts.append(x ** (2 * k + 1) / math.factorial(2 * k + 1))
    return math.fsum(parts)


if __name__ == "__main__":
    x = float(input("Введите x от -1 до 1: "))

    if x < -1 or x > 1:
        print("Недопустимое значение: x должен быть от -1 до 1", file=sys.stderr)
        sys.exit(1)

    result, count, remainder = sinh_series(x)

    print(f"sinh({x}) = {result:.10f}")
    print(f"Членов ряда: {count}")
    print(f"Оценка остатка: {remainder:.3e}")

    check = sinh_check(x)
    print(f"Проверочный расчёт: {check:.10f}")
    print(f"Расхождение: {abs(result - check):.3e}")
    print(f"math.sinh: {math.sinh(x):.10f}")
```

Таблица 5. Проверки программы individual_04.py

Входные данные (по одному на строку)	Ожидаемый результат	Фактический результат	Вывод
0	sinh(0.0) = 0.0000000000 Членов ряда: 1 Оценка остатка: 0.000e+00	sinh(0.0) = 0.0000000000 Членов ряда: 1 Оценка остатка: 0.000e+00	Совпадает
0.5	sinh(0.5) = 0.5210953055 Членов ряда: 6 Оценка остатка: 1.223e-11	sinh(0.5) = 0.5210953055 Членов ряда: 6 Оценка остатка: 1.223e-11	Совпадает
-0.5	sinh(-0.5) = -0.5210953055 Членов ряда: 6 Оценка остатка: 1.223e-11	sinh(-0.5) = -0.5210953055 Членов ряда: 6 Оценка остатка: 1.223e-11	Совпадает
1	sinh(1.0) = 1.1752011936 Членов ряда: 8 Оценка остатка: 7.647e-13	sinh(1.0) = 1.1752011936 Членов ряда: 8 Оценка остатка: 7.647e-13	Совпадает
-1	sinh(-1.0) = -1.1752011936 Членов ряда: 8 Оценка остатка: 7.647e-13	sinh(-1.0) = -1.1752011936 Членов ряда: 8 Оценка остатка: 7.647e-13	Совпадает
1.5	Ошибка	Ошибка: Недопустимое значение: x должен быть от -1 до 1 (код завершения 1)	Совпадает

Таблица 6. Независимая проверка (выводится программой после основного вычисления)

x	Ряд (основной расчёт)	Проверочный расчёт (50 членов)	Расхождение	math.sinh
0	0.0000000000	0.0000000000	0.000e+00	0.0000000000
0.5	0.5210953055	0.5210953055	1.965e-14	0.5210953055
-0.5	-0.5210953055	-0.5210953055	1.965e-14	-0.5210953055
1	1.1752011936	1.1752011936	2.665e-15	1.1752011936
-1	-1.1752011936	-1.1752011936	2.665e-15	-1.1752011936

Расхождение основного расчёта и независимой суммы не превышает $3 \cdot 10^{-15}$, что много меньше $\varepsilon = 10^{-10}$; значения до 10 знаков совпадают с

math.sinh. Функция нечётна: значения при x и −x отличаются только знаком. При x = 0 первый член равен нулю, цикл не выполняется, получаем 1 член и сумму 0. Значение x = 1.5 вне диапазона отклоняется с кодом 1.

Таблица 7. Трассировка программы individual_04.py при x = 0.5

k	Коэффициент q	Член a_k	Накопленная сумма	Оценка остатка |a_k|	|a_k| > ε
0	—	0.5000000000	0.5000000000	5.000e-01	да
1	$x^2/(2\cdot3)$	0.0208333333	0.5208333333	2.083e-02	да
2	$x^2/(4\cdot5)$	0.0002604167	0.5210937500	2.604e-04	да
3	$x^2/(6\cdot7)$	0.0000015501	0.5210953001	1.550e-06	да
4	$x^2/(8\cdot9)$	0.0000000054	0.5210953055	5.382e-09	да
5	$x^2/(10\cdot11)$	0.0000000000	0.5210953055	1.223e-11	нет

Три первых включённых члена: $a_0 = 0.5$, $a_1 = 0.5^3/3! \approx 0.0208333333$, $a_2 = 0.5^5/5! \approx 0.0002604167$. Итого включено 6 членов, последний член $a_5 \approx 1.223\cdot10^{-11} \le \varepsilon$, поэтому цикл завершился; накопленная сумма 0.5210953055 совпадает с независимым расчётом (расхождение $2\cdot10^{-14}$).

Наблюдение в отладчике. Точка останова на строке term *= x * x / ((2 * k + 2) * (2 * k + 3)) при x = 0.5. При первой остановке k = 0, term = 0.5, total = 0.5, count = 1; следующим шагом term становится 0.0208333333…, затем k = 1, total = 0.5208333333…, count = 2. Остановка на проверке while abs(term) > EPSILON после пятого шага (k = 5): term ≈ 1.223e-11 меньше EPSILON, цикл завершается, управление переходит к return total, count, abs(term).

Контрольные вопросы

1. Чем различаются присваивание и сравнение?

Ответ: Присваивание (=) записывает значение в переменную и не даёт результата-условия; сравнение (==) проверяет равенство и возвращает значение типа bool.

2. Какие значения считаются ложными? Почему строка "0" истинна?

Ответ: Ложными считаются False, None, числа 0 и 0.0, пустые строки и пустые коллекции. Строка "0" непустая, а для строк важно только наличие символов, а не их смысл, поэтому она истинна.

3. Как работают and, or, not и сокращённое вычисление?

Ответ: and истинно, если истинны оба операнда, or — если истинен хотя бы один, not меняет значение на противоположное. При сокращённом вычислении правый операнд не вычисляется, если результат уже определён левым (ложное для and, истинное для or).

4. Чем несколько if отличается от цепочки if–elif–else?

Ответ: Отдельные if проверяются независимо, поэтому могут выполниться несколько веток. В цепочке выполняется только первая ветка с истинным условием, остальные не проверяются.

5. Почему порядок условий важен при пересекающихся диапазонах?

Ответ: Проверка идёт сверху вниз, и срабатывает первое истинное условие. Если поставить более широкое условие раньше узкого, узкое никогда не выполнится. В задании 1 условия идут по возрастанию границ 50, 75, 90.

6. Что такое условное выражение и когда оно удобно?

Ответ: Это запись вида значение_1 if условие else значение_2. Она удобна, когда нужно выбрать одно из двух значений для присваивания, например bonus = 20 if kg > 90 else 0.

7. С какой версии Python доступен match?

Ответ: Оператор match–case доступен начиная с Python 3.10.

8. Чем числовой образец 1 отличается от строкового "1"?

Ответ: Образец 1 сопоставляется с числом, а "1" — со строкой. Результат input() — строка, поэтому без int() числовой образец не совпадёт.

9. Что означает case _ и почему он размещается последним?

Ответ: case _ — образец, подходящий любому значению (ветка по умолчанию). Он стоит последним, потому что перехватывает всё и ветки после него не проверялись бы.

10. Как объединить несколько литералов в одной ветви case?

Ответ: С помощью вертикальной черты: case 1 | 2 | 3:.

11. Нужен ли break после case? Что произойдёт при отсутствии совпадений?

Ответ: break не нужен: выполняется только первая совпавшая ветка, после чего match завершается. Если совпадений нет и case _ отсутствует, не выполняется ни одна ветка.

12. Почему case name не сравнивает значение с переменной name?

Ответ: Простое имя в образце — захватывающий образец: оно совпадает с любым значением и связывает его с этим именем. Для сравнения используют точечное имя (Color.RED) или условие guard: case v if v == name.

13. Когда удобнее использовать if, а когда простейший match?

Ответ: if удобен для диапазонов и составных условий с разными переменными. match удобен, когда одно выражение сравнивается с набором конкретных значений.

14. Как работает for и что возвращает range?

Ответ: for поочерёдно присваивает переменной элементы итерируемого объекта. range(a, b, step) выдаёт числа от a до b − 1 с шагом step (правая граница не включается).

15. Как получить 15, 13, …, 1? Почему ноль в этой последовательности отсутствует?

Ответ: range(15, 0, -2). Значения нечётные: от 15 они уменьшаются на 2 и доходят до 1; следующее значение −1 уже вне границы. Ноль чётный, а начав с 15 с шагом 2, до него не дойти, и правая граница 0 не включается.

16. Когда while не выполнит ни одной итерации?

Ответ: Когда условие ложно при первой проверке, например while x > 0 при x = 0.

17. Чем отличаются break и continue? На какой вложенный цикл они действуют?

Ответ: break полностью прекращает цикл, continue переходит к следующей итерации. Оба действуют на ближайший объемлющий (внутренний) цикл.

18. Как continue может привести к бесконечному циклу?

Ответ: Если в цикле while continue стоит до изменения счётчика, то счётчик не изменяется, условие остаётся истинным и цикл не заканчивается.

19. Когда выполняется else у цикла?

Ответ: Когда цикл завершился естественно, то есть условие стало ложным или перебор закончился, и без выхода через break.

20. Как выбирается начальное значение накопителя суммы и произведения?

Ответ: Для суммы — 0, для произведения — 1, то есть нейтральный элемент операции. В ряде в накопитель сразу кладётся первый член a_0, как в задании 4.

21. Чем условие остановки численного алгоритма отличается от оценки погрешности?

Ответ: Условие остановки определяет, когда цикл заканчивается (abs(term) ≤ ε). Оценка погрешности показывает, насколько отброшенный остаток может отличаться от результата; условие остановки должно её гарантировать, что доказывается неравенством $|a_{k+1}| \le ½|a_k|$.

22. Почему приближённые вещественные результаты иногда сравнивают через math.isclose?

Ответ: Числа float хранятся с округлением, поэтому точное сравнение == ненадёжно. math.isclose сравнивает значения с допустимой относительной и абсолютной погрешностью.

23. Как составить тесты для проверки границ ветвления?

Ответ: Для каждой границы берут значение на самой границе и значения слева и справа от неё, а также внутри диапазона и за допустимыми пределами. В задании 1 это 50, 50.5, 75, 75.5, 90, 90.5 и -1, 1000.1.

24. Что показывает трассировочная таблица и чем она дополняет конечный вывод?

Ответ: Она показывает значения переменных на каждом шаге цикла. Конечный вывод говорит только об итоге, а трассировка подтверждает ход алгоритма и помогает найти ошибку в промежуточных значениях.

25. Чем отличаются stdout, stderr и код завершения? Почему сообщение об ошибке не останавливает программу автоматически?

Ответ: stdout — стандартный вывод результатов, stderr — поток для сообщений об ошибках; код завершения 0 означает успех, ненулевой — ошибку. print(..., file=sys.stderr) только выводит текст, а завершает программу вызов sys.exit(1).

Вывод

В ходе работы выполнены четыре индивидуальных задания варианта 13: выбор тарифа цепочкой if-elif-else, проверка делимости, перебор делителей циклом while и вычисление гиперболического синуса суммированием ряда с точностью 10^{-10}. Для каждой программы составлены таблицы проверок с граничными и недопустимыми значениями, для заданий 3 и 4 приведены трассировочные таблицы. Результат ряда подтверждён независимым расчётом и функцией math.sinh, расхождение не превышает $3 \cdot 10^{-15}$.